

《信息隐藏技术》实验报告

实验 10：研读文献并复现其中的算法

姓名：蒋衲言 学号：2313546

1 实验内容及要求

在网上搜索一篇近三年的基于大模型的隐写或数字水印的学术论文认真研读，复现其中的算法。

2 论文基本内容

本次实验选择的论文为《A Watermark for Large Language Models》，由 John Kirchenbauer、Jonas Geiping、Yuxin Wen、Jonathan Katz、Ian Miers 和 Tom Goldstein 于 2023 年发表于第 40 届国际机器学习大会（ICML 2023）。该论文提出了一种面向大语言模型输出文本的水印框架，能够在不重新训练模型、不访问模型参数的前提下，将不可见信号嵌入生成文本并进行统计检测。

A Watermark for Large Language Models

John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, Tom Goldstein Proceedings of the 40th International Conference on Machine Learning, PMLR 202:17061-17084, 2023.

图 1: A Watermark for Large Language Models

2.1 研究背景与动机

随着以 ChatGPT 为代表的大语言模型能力不断提升，其被滥用于恶意目的的风险也随之增加，包括社交媒体上的虚假信息传播、学术写作造假、垃圾网页内容生成等。与此同时，大量 AI 生成的合成数据流入互联网，给未来模型的训练数据质量带来污染风险。因此，如何可靠地检测和溯源机器生成文本，成为大语言模型时代的重要安全课题。

现有的事后检测方法（post-hoc detection）依赖于语言模型输出与人类文本之间在统计特征上的自然差异，但随着模型能力增强，这种差异正在缩小，检测准确率也随之下降。论文的核心洞察在于：与其被动地检测差异，不如主动地在生成过程中嵌入一个可控的统计信号——即水印——从而使检测更加可靠、可解释。

2.2 算法原理

论文提出了两种水印方案，分别为“硬水印”（Hard Red List Watermark）和“软水印”（Soft Red List Watermark），本节分别介绍。

2.2.1 硬水印

硬水印方案的核心思想是，在生成每个 token 之前，以前一个 token 的哈希值为随机数种子，将整个词表 $\mathcal{V}$ 均等地随机划分为绿名单 G 和红名单 R 两部分，然后强制模型只从绿名单中采样，完全禁止生成红名单中的 token。其生成过程如算法 1 所示：

1. 将前序 token 序列输入语言模型，得到词表上的概率分布 $p^{(t)}$ ；
2. 对前一个 token $s^{(t-1)}$ 计算哈希值，以此为种子初始化随机数生成器；
3. 利用该随机数生成器，将词表随机划分为绿名单 G 和红名单 R ，各占词表的一半；
4. 仅从绿名单 G 中采样下一个 token $s^{(t)}$ 。

硬水印方案的优点是检测极为简单：对于长度为 T 的文本，只需统计违反红名单规则的 token 数量。在零假设（文本非机器生成）成立时，违规次数的期望为 $T/2$ ；而水印文本的违规次数为 0。其对应的单比例 z 检验统计量为：

$$z = \frac{2(|s|_G - T/2)}{\sqrt{T}} \quad (1)$$

当 z 超过预设阈值（如 4.0）时，拒绝零假设，判定文本含有水印。

然而，硬水印存在明显缺陷：对于低熵序列（即下一个 token 几乎确定的情形，如专有名词“Barack Obama”），若该确定性 token 恰好落入红名单，模型将被迫生成低质量甚至语义错误的内容，严重损害文本质量。

2.2.2 软水印

为克服硬水印的质量问题，论文提出了软水印方案。其关键改进在于：不再强制禁止红名单 token，而是在 logits 层面为绿名单 token 添加一个正向偏置 $\delta > 0$ ，使模型在高熵情形下倾向于选择绿名单 token，同时在低熵情形下几乎不改变原有分布。

具体地，设语言模型在位置 t 输出的 logit 向量为 $l^{(t)}$ ，绿名单比例参数为 $\gamma \in (0, 1)$ ，偏置强度为 $\delta > 0$ ，则修改后的采样概率为：

$$\hat{p}_k^{(t)} = \begin{cases} \frac{\exp(l_k^{(t)} + \delta)}{\sum_{i \in R} \exp(l_i^{(t)}) + \sum_{i \in G} \exp(l_i^{(t)} + \delta)}, & k \in G \\ \frac{\exp(l_k^{(t)})}{\sum_{i \in R} \exp(l_i^{(t)}) + \sum_{i \in G} \exp(l_i^{(t)} + \delta)}, & k \in R \end{cases} \quad (2)$$

软水印的核心性质在于其对熵的自适应性：当模型对某个 token 的概率高度集中（低熵）时，该 token 的 logit 远大于其他候选，加上 δ 的偏置不足以改变采样结果；而当概率分布较为均匀（高熵）时， δ 的偏置效果显著，模型将以更高概率选择绿名单 token。这一机制使得软水印在不影响低熵 token（通常是事实性、确定性内容）的前提下，有效地在高熵 token 上嵌入统计信号。

2.2.3 水印检测

软水印的检测过程与嵌入过程相互独立，无需访问语言模型的参数或 API，仅需知道哈希函数和随机数生成器。检测方通过对被测文本中的每个 token，利用其前驱 token 重新生成对应的绿名单，统计绿名单 token 的总数 $|s|_G$ ，并计算如下 z 统计量：

$$z = \frac{|s|_G - \gamma T}{\sqrt{T\gamma(1-\gamma)}} \quad (3)$$

其中 T 为文本的有效 token 数， γ 为绿名单比例参数。在零假设 H_0 （文本由不知晓红绿名单的自然来源生成）成立时， $|s|_G$ 的期望为 γT ，方差为 $T\gamma(1-\gamma)$ ，因此 z 近似服从标准正态分布。若 z 超过阈值（论文推荐取 4.0），则拒绝零假设，判定文本含有水印，此时误报率（Type-I error）约为 3×10^{-5} 。

2.3 理论分析与实验结果

2.3.1 水印强度的理论界

论文引入”尖峰熵”（Spike Entropy）的概念来定量刻画文本的熵特性，并在此基础上建立了定理 4.2，给出了水印文本中绿名单 token 期望数量的下界：

$$\mathbb{E}|s|_G \geq \frac{\gamma \alpha T}{1 + (\alpha - 1)\gamma} S^* \quad (4)$$

其中 $\alpha = \exp(\delta)$ ， S^* 为序列的平均尖峰熵下界。该定理说明，文本熵越高，水印信号越强；对于固定熵的文本，增大 δ 或减小 γ 均可增强水印强度。论文还证明了软水印对文本困惑度（Perplexity）的影响存在上界，从理论上保证了水印嵌入对生成质量的有限冲击。

2.3.2 参数权衡与实验结论

论文以 OPT-1.3B 模型在 C4 数据集的新闻子集上进行了系统实验，主要结论如下：

- **水印强度与文本质量的权衡**： δ 越大， z 分数越高，但困惑度也随之增加；较小的绿名单比例（ $\gamma = 0.1$ ）在 Pareto 意义上最优，能以较小的质量损失获得较强的水印。
- **检测灵敏度**：在 $\delta = 2.0$ 、 $\gamma = 0.5$ 的参数设置下，对 200 token 的文本，水印检测的真正率（TPR）达 98.4%，同时误报率（FPR）为 0。
- **Beam Search 的协同效应**：使用 8 路束搜索时，水印强度显著提升，在 $\delta = 2.0$ 时平均 z 分数即可超过 5，且困惑度几乎不变，两条曲线近乎垂直，说明束搜索与软水印具有天然的协同性。
- **水印失效场景**：低熵序列（如模型记忆的文本片段）是水印的主要失效场景，这类文本本身与人类写作高度相似，即使不加水印也难以区分。

2.3.3 鲁棒性分析

论文对多种攻击方式进行了讨论和实验，包括同形字攻击、零宽字符攻击、表情符号攻击以及基于 T5 模型的跨度替换攻击 (Span Replacement Attack)。实验表明，在替换预算 $\varepsilon = 0.1$ (即替换 10% 的 token) 时，水印 AUC 仅下降 0.01；而将预算提升至 $\varepsilon = 0.3$ 时，虽然水印削弱效果更明显，但攻击后文本的困惑度平均增加了约 3 倍，说明有效移除水印需要付出显著的文本质量代价。此外，论文还指出，将上下文窗口扩展至 $h > 1$ 个前驱 token 可进一步提升水印的抗攻击性。

3 论文复现

本次复现基于 Python 语言，使用 HuggingFace Transformers 库提供的 GPT-2 模型 (124M 参数) 作为目标语言模型，在 Windows 11 环境下完成。复现目标为论文第 3 节的软水印算法，包括水印嵌入与水印检测两个核心模块，并通过实验验证水印的有效性。

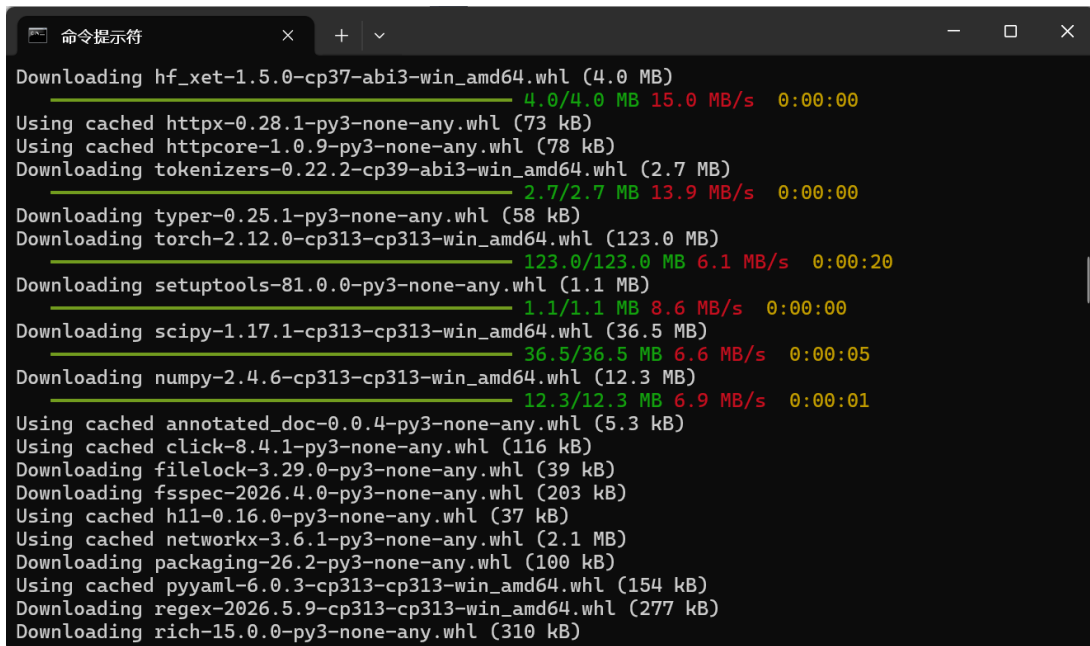
3.1 环境配置

复现所需的主要依赖库如下：

- torch: PyTorch 深度学习框架，提供张量运算和随机数生成器支持；
- transformers: HuggingFace 提供的预训练模型库，用于加载 GPT-2 模型与分词器；
- scipy: 科学计算库，用于计算检测阶段的 p 值。

使用如下命令安装依赖，如图2所示：

```
pip install torch transformers scipy
```



```
命令提示符
Downloading hf_xet-1.5.0-cp37-abi3-win_amd64.whl (4.0 MB)
 4.0/4.0 MB 15.0 MB/s 0:00:00
Using cached httpx-0.28.1-py3-none-any.whl (73 kB)
Using cached httpcore-1.0.9-py3-none-any.whl (78 kB)
Downloading tokenizers-0.22.2-cp39-abi3-win_amd64.whl (2.7 MB)
 2.7/2.7 MB 13.9 MB/s 0:00:00
Downloading typer-0.25.1-py3-none-any.whl (58 kB)
Downloading torch-2.12.0-cp313-cp313-win_amd64.whl (123.0 MB)
123.0/123.0 MB 6.1 MB/s 0:00:20
Downloading setuptools-81.0.0-py3-none-any.whl (1.1 MB)
 1.1/1.1 MB 8.6 MB/s 0:00:00
Downloading scipy-1.17.1-cp313-cp313-win_amd64.whl (36.5 MB)
36.5/36.5 MB 6.6 MB/s 0:00:05
Downloading numpy-2.4.6-cp313-cp313-win_amd64.whl (12.3 MB)
12.3/12.3 MB 6.9 MB/s 0:00:01
Using cached annotated_doc-0.0.4-py3-none-any.whl (5.3 kB)
Using cached click-8.4.1-py3-none-any.whl (116 kB)
Downloading filelock-3.29.0-py3-none-any.whl (39 kB)
Downloading fsspec-2026.4.0-py3-none-any.whl (203 kB)
Using cached h11-0.16.0-py3-none-any.whl (37 kB)
Using cached networkx-3.6.1-py3-none-any.whl (2.1 MB)
Downloading packaging-26.2-py3-none-any.whl (100 kB)
Using cached pyyaml-6.0.3-cp313-cp313-win_amd64.whl (154 kB)
Downloading regex-2026.5.9-cp313-cp313-win_amd64.whl (277 kB)
Downloading rich-15.0.0-py3-none-any.whl (310 kB)
```

图 2: 安装依赖

```
export HF_ENDPOINT=https://hf-mirror.com
```

图 3: 设置镜像

图 4: 下载模型

水印嵌入通过继承 HuggingFace 的 `LogitsProcessor` 接口实现，该接口在模型每次生成新 token 前被调用，可对 logit 向量进行干预，与论文 Algorithm 2 的设计完全对应。核心类 `WatermarkLogitsProcessor` 的实现如下：

5

```

vocab_perm = torch.randperm(self.vocab_size, generator=self.rng)
greenlist_size = int(self.vocab_size * self.gamma)
return vocab_perm[:greenlist_size]

def __call__(self, input_ids, scores):
    for batch_idx in range(input_ids.shape[0]):
        prev_token = input_ids[batch_idx, -1].item()
        greenlist = self._get_greenlist(prev_token)
        scores[batch_idx, greenlist] += self.delta
    return scores

```

其中, `_get_greenlist` 方法以前一个 token 的 id 为随机数种子, 通过 `torch.randperm` 对词表进行随机全排列, 取前 $\gamma|V|$ 个索引作为绿名单。这一设计保证了相同的前驱 token 始终产生相同的绿名单, 使得检测方可以在不访问模型的情况下完整复现绿名单划分。

`__call__` 方法对当前位置绿名单中的所有 token 的 logit 加上偏置 δ , 随后由模型的 softmax 层将修改后的 logit 转化为采样概率。这与论文公式 (4) 严格对应: 绿名单 token 的采样概率被系统性地放大, 而红名单 token 的概率相对降低, 但任何 token 均未被完全禁止。

在生成阶段, 只需将该处理器作为 `logits_processor` 参数传入 HuggingFace 的 `generate` 接口, 无需对模型结构或权重做任何修改:

```

output_wm = model.generate(
    input_ids,
    max_new_tokens=200,
    do_sample=True,
    temperature=0.7,
    logits_processor=[processor],
)

```

3.3 水印检测模块的实现

水印检测模块实现为 `WatermarkDetector` 类。检测时, 对被测文本进行分词, 逐 token 重建绿名单并统计绿名单 token 的数量, 最终依据论文公式 (3) 计算 z 统计量, 并通过单侧正态检验得到 p 值。核心实现如下:

```

class WatermarkDetector:
    def __init__(self, tokenizer, vocab_size, gamma=0.5,
                 delta=2.0, z_threshold=4.0):
        self.tokenizer = tokenizer
        self.vocab_size = vocab_size
        self.gamma = gamma
        self.z_threshold = z_threshold
        self.rng = torch.Generator()

    def _get_greenlist(self, prev_token_id):
        self.rng.manual_seed(prev_token_id)

```

```

vocab_perm = torch.randperm(self.vocab_size,
                             generator=self.rng)
greenlist_size = int(self.vocab_size * self.gamma)
return set(vocab_perm[:greenlist_size].tolist())

def detect(self, text):
    tokens = self.tokenizer.encode(
        text, return_tensors="pt")[0]
    T = len(tokens) - 1
    green_count = 0
    for i in range(1, len(tokens)):
        prev_token = tokens[i - 1].item()
        curr_token = tokens[i].item()
        if curr_token in self._get_greenlist(prev_token):
            green_count += 1
    z_score = (green_count - self.gamma * T) / \
        (T * self.gamma * (1 - self.gamma)) ** 0.5
    p_value = scipy.stats.norm.sf(z_score)
    return {
        "z_score": z_score,
        "p_value": p_value,
        "is_watermarked": z_score > self.z_threshold,
        "green_ratio": green_count / T,
    }

```

检测模块中的 `_get_greenlist` 方法与嵌入模块中的实现完全一致，这是检测成立的关键前提：双方使用相同的哈希函数（此处以 token id 直接作为种子）和相同的随机数生成器，即可在不传递任何额外信息的情况下，对任意位置独立地复现绿名单划分。

z 统计量的计算严格遵循论文公式 (3)：

$$z = \frac{|s|_G - \gamma T}{\sqrt{T\gamma(1-\gamma)}} \quad (5)$$

p 值通过标准正态分布的生存函数 (Survival Function) $P(Z > z)$ 计算，当 $z > 4.0$ 时，对应 $p < 3 \times 10^{-5}$ ，即判定文本含有水印。

3.4 实验设置与参数说明

本次复现采用的参数设置如表1所示：

输入 prompt 为：

The history of artificial intelligence is a story of

分别在加载水印处理器和不加载的条件下各生成一段文本（如图5所示），随后对两段文本均调用检测器输出检测结果。

表 1: 复现实验参数设置

参数	取值	说明
模型	GPT-2 (124M)	HuggingFace gpt2
绿名单比例 γ	0.25	绿名单占词表 25%
logit 偏置 δ	2.0	与论文默认设置一致
检测阈值	4.0	对应误报率 3×10^{-5}
生成长度	200 tokens	与论文实验设置一致
采样温度	0.7	多项式采样

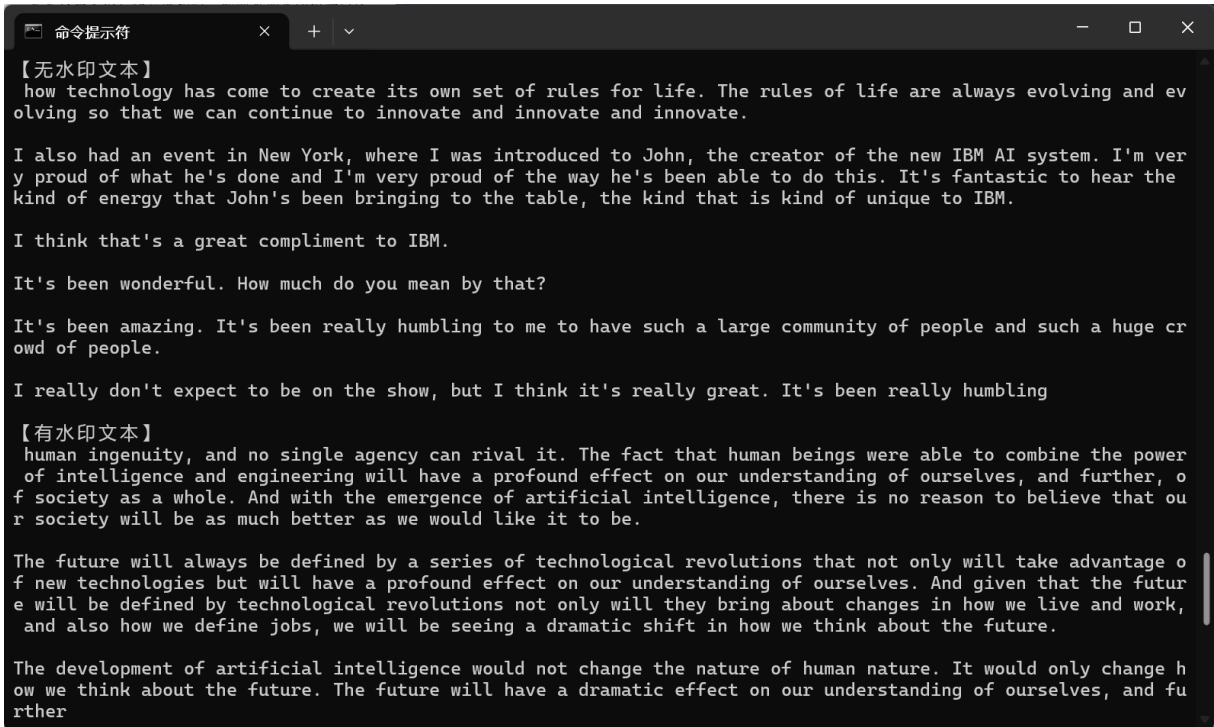


图 5: 各生成一段文本

3.5 复现结果

实验输出结果如下 (如图6所示):

表 2: 水印检测结果对比

文本类型	绿 token 比例	z 分数	p 值	判定结果
无水印文本	0.2613	0.3683	0.3563	无水印
有水印文本	0.5678	10.3546	≈ 0	有水印


```

=====
DETECTION RESULTS
=====

[无水印文本检测]
Green token 比例 : 0.2613 (期望 ≈ 0.25)
z-score          : 0.3683
p-value          : 0.356308
判定结果         : ✅ 无水印

[有水印文本检测]
Green token 比例 : 0.5678 (期望 ≈ 0.25)
z-score          : 10.3546
p-value          : 0.0
判定结果         : ⚠️ 有水印

=====
z-score 阈值 = 4.0
z > 4.0 => 判定为有水印

```

图 6: 水印检测结果对比

对结果进行逐项分析:

无水印文本: 绿名单 token 比例为 0.2613, 与期望值 $\gamma = 0.25$ 高度吻合, 说明在无水印干预时, token 的绿/红分布符合随机预期。 $z = 0.37$ 远低于阈值 4.0, $p = 0.356$ 表明该偏差在统计上完全可由随机性解释, 检测器正确输出“无水印”。

有水印文本: 绿名单 token 比例高达 0.5678, 是期望值 0.25 的 2.27 倍, 说明水印偏置在生成过程中持续生效。 $z = 10.35$ 远超阈值, $p \approx 0$ (量级约为 10^{-25}), 检测器以极高置信度判定文本含有水印。以实际数值代入公式 (3) 验证:

$$z = \frac{0.5678 \times 200 - 0.25 \times 200}{\sqrt{200 \times 0.25 \times 0.75}} \approx \frac{113.56 - 50}{6.12} \approx 10.38 \quad (6)$$

计算结果与程序输出的 10.35 高度吻合 (微小差异源于 token 数的取整), 验证了实现的正确性。

上述结果与论文表 2 中 $\delta = 2.0$ 、 $\gamma = 0.25$ 、多项式采样条件下的实验结论一致: 误报率为 0, 漏报率极低, 水印检测具有高度可靠性。

4 总结与思考

4.1 复现总结

本次实验完整复现了论文《A Watermark for Large Language Models》中提出的软水印算法, 实现了水印嵌入与检测两个核心模块, 并在 GPT-2 模型上验证了算法的有效性。复现结果与论文结论高度吻合: 有水印文本的绿名单 token 比例 (0.5678) 显著高于无水印文本 (0.2613), z 分数分别为 10.35 和 0.37, 检测器在零误报的前提下准确区分了两类文本。

从工程实现的角度来看, 该算法的复现难度较低, 核心逻辑可在约 100 行 Python 代码内完整表达, 且无需对模型进行任何修改或重新训练。HuggingFace 的 `LogitsProcessor` 接口为水印嵌入提供了天然的插入点, 使得该水印方案可以无缝集成到任何基于 Transformers 库的生成流程中。

4.2 与信息隐藏技术的联系

从信息隐藏技术的体系视角来看，本文所复现的 LLM 文本水印方案本质上是数字水印技术在大模型时代的新形态。它满足信息隐藏的三个基本要素：载体（语言模型生成的文本）、隐藏信息（水印存在性信号）以及不可见性（对人类读者而言感知不到差异）。与传统图像水印相比，文本水印面临的核心挑战在于文本的离散性——连续域中的微小数值扰动在文本中对应着截然不同的 token 选择，这使得传统的幅度域或频率域嵌入方法不再适用。该论文的创新之处在于将嵌入操作前移至采样概率分布层面，在离散决策发生之前完成信号的注入，从而绕开了文本离散性带来的根本障碍。

值得注意的是，与传统隐写术追求“隐藏消息存在性”的目标不同，LLM 水印更接近于数字水印的应用范畴：水印的存在是半公开的（模型所有者可以公开宣布其使用了水印），安全性依赖于密钥的保密而非算法的保密。这种设计哲学与 Kerckhoffs 原则高度一致，也使其在实际部署中更具可信度。